Near-Circle

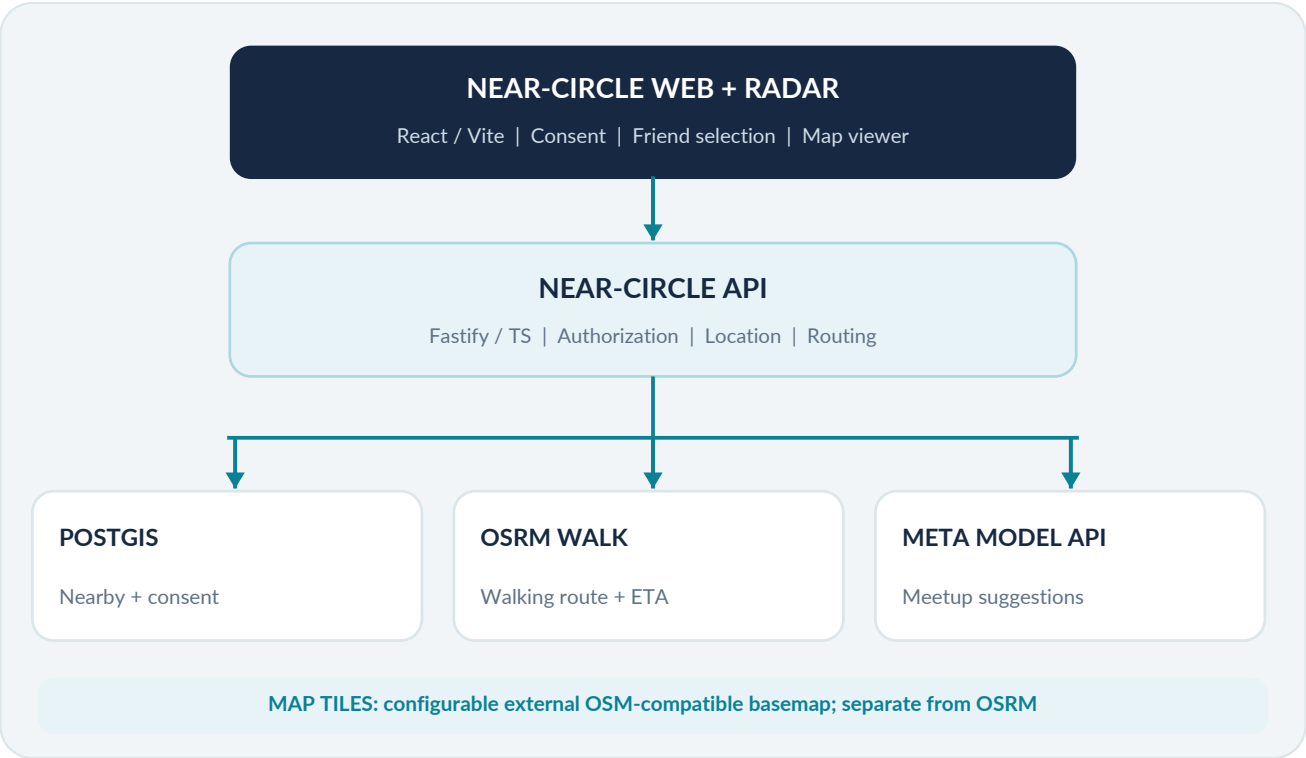
A Messenger-style prototype that turns opt-in friend proximity into an interactive radar - then a real walking route, ETA and estimated steps.

MESSENGER-STYLE PROTOTYPE

REAL GEOSPATIAL ENGINE

META AI DEMO

01 / SYSTEM ARCHITECTURE



02 / MVP OUTCOMES

Opt-in radar discovery

Eligible nearby friends appear as red dots around the current user.

Route on selection

Select a friend to request consent-checked walking directions and step estimates.

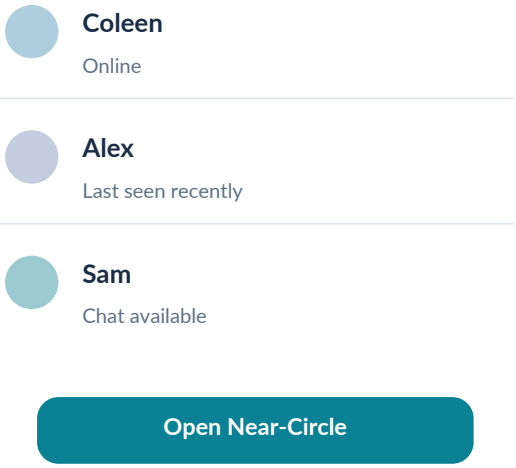
Real engine, honest demo

PostGIS and walking OSRM are real; social accounts and messages are seeded.

Four-screen user journey

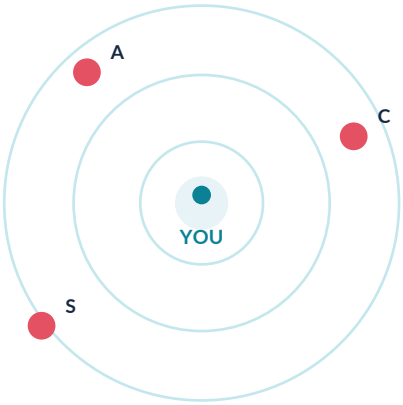
A familiar messaging entry point, an opt-in radar, a selected friend and a genuine walking route. All interface identities are simulated.

01 / Messenger-style inbox



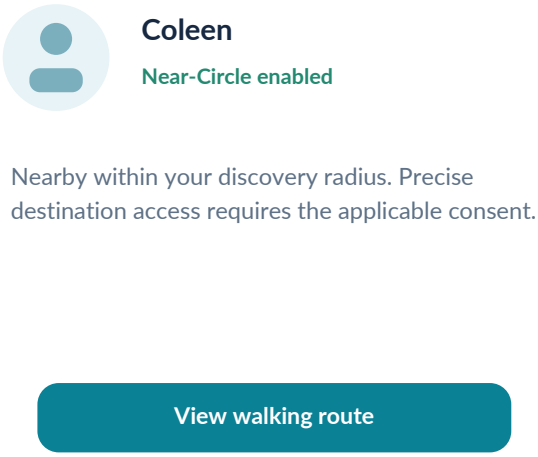
CLICK OPEN NEAR-CIRCLE

02 / Radar



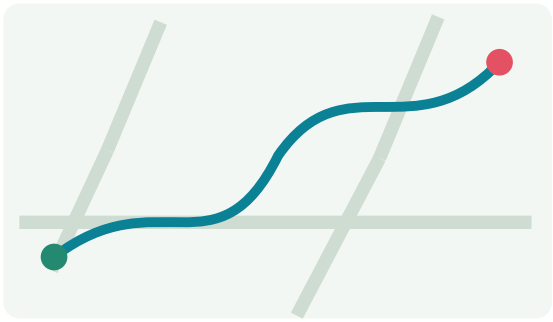
TAP A RED FRIEND DOT

03 / Friend selected



CHOOSE VIEW WALKING ROUTE

04 / Route + steps



900 m ~12 min walk

~1,200 estimated steps

REVIEW WALKING DIRECTIONS

Radar rule: user centered; only eligible, opted-in friends appear, positioned using distance and bearing. Route values above are illustrative; the service provides actual results.

04 / DELIVERY BOUNDARIES

Real engine. Mock social layer.

The demo proves permission-aware proximity and walking routing without claiming Facebook access or approval.

IMPLEMENT FOR REAL

- **PostGIS**
  Indexed ST_DWithin, radius queries
- **Consent enforcement**
  Visibility, friendships, freshness
- **Walking OSRM**
  Foot-profile geometry, time, distance
- **Street-map renderer**
  Configurable tiles + attribution
- **Steps and AI**
  Stride estimate + Meta Model API

SIMULATE FOR THE DEMO

- **Messenger shell**
  Unofficial, labeled user interface
- **Friend identities**
  Seed users and conversations
- **Friend graph**
  Application-defined connections
- **Location data**
  Clearly simulated GPS positions
- **Chat and notifications**
  Demonstration only; no live sending

CORE LOCATION + ROUTE LOGIC

01 / SEARCH	ST_DWithin on PostGIS geography points
02 / FILTER	Opt-in, friends, permitted visibility, freshness
03 / RADAR	Bearing + distance position the red dots
04 / ROUTE	Authorized request to a pedestrian OSRM server
05 / STEPS	Round(route_distance_m / stride_m)

Routing constraint: a car-profile OSRM graph is not a pedestrian router. Prepare and test a dedicated **foot.lua** graph before claiming walking routes or ETAs.

01 / SERVICE TOPOLOGY

Independent services. One project.

Four separately managed local services; one configurable external map-tile dependency. All project-owned services can run on one development PC.

01 FRONTEND SERVICE

React / Vite

Messenger-style inbox, red-dot radar, consent toggle, friend card, Leaflet or MapLibre map.

02 BACKEND API

Fastify / TypeScript

Demo identity, visibility controls, scoped location access, walking-route adapter, AI gateway.

03 DATABASE

PostgreSQL + PostGIS

Friend graph, geography points, expiration timestamps, radius queries and permissions.

04 WALKING ROUTER

OSRM / foot.lua

Independent HTTP route API, prepared pedestrian graph, own health checks and lifecycle.

EXTERNAL LOGICAL DEPENDENCY

OSM-COMPATIBLE MAP TILE PROVIDER

Supplies the street basemap to the browser with proper attribution. It is **not** the OSRM router and is **not** started as a fifth local container for the MVP.

DEPLOYMENT CONTRACT

- One independent Near-Circle repository and Docker Compose configuration.
- No runtime dependency on unrelated local applications or routing instances.
- Only the Fastify API connects the browser to protected friend routes.
- OSRM remains directly testable, even while the frontend/API are stopped.

04 / WALKING ROUTE INTEGRATION

A route is earned by consent.

Route access is a second authorization decision, not a side effect of seeing a friend on radar.

01 — SELECT

User chooses a visible friend on the radar.

02 — RECHECK

Fastify checks relationship, consent, radius and freshness.

03 — ROUTE

Adapter requests OSRM pedestrian geometry, distance and duration.

04 — NORMALIZE

Validate response, reject timeout / no-route / wrong graph.

05 — ESTIMATE

Steps = round(distance_m / stride_m); default 0.75 m.

06 — RENDER

Show street map, permitted route, labeled estimated ETA/steps.

ROUTING ADAPTER INTERFACE / ILLUSTRATIVE

```
interface WalkingRoutingProvider {
  route({ from, to }): Promise<{
    geometry: GeoJSON.LineString;
    distance_m: number; duration_s: number;
  }>;
}
```

PEDESTRIAN PROFILE REQUIREMENT

Prepare a standalone OSRM graph with **foot.lua** from suitably licensed OSM data. Graph extraction, partitioning/customization and validation happen as an explicit first-time bootstrap process, never automatically at every startup.

Maps and routing stay separate.

Configuration, attribution and location consent are first-class requirements - not shortcuts for the demo.

ILLUSTRATIVE ENVIRONMENT CONFIGURATION

```
ROUTING_PROVIDER=osrm

ROUTING_WALK_URL=http://near-circle-routing-walk:5000

MAP_TILE_URL=https://tile.openstreetmap.org/{z}/{x}/{y}.png

MAP_TILE_ATTRIBUTION=© OpenStreetMap contributors

DEFAULT_STRIDE_M=0.75
```

MAP PROVIDER USAGE CONDITIONS

- **Browser basemap**
  Leaflet / MapLibre gets tiles from a configurable external provider.
- **Attribution is visible**
  Credit OSM contributors and obey chosen tile provider policies.
- **No bulk downloading**
  Respect caching/identification policies; do not prefetch offline tiles.
- **Future self-hosting**
  Use supported tile infrastructure if demo traffic or offline use demands it.

PERMISSION AND DATA HANDLING GATES

- [ ] Hide disabled, unapproved or stale friends from radar AND route endpoints.
- [ ] Reject the route request if consent was revoked after discovery.
- [ ] Never trust a client-supplied exact coordinate as friend-location authority.
- [ ] Validate coordinate bounds, OSRM failures and no-route results honestly.
- [ ] Never send exact friend coordinates to Meta AI without necessity and consent.

04 / UNIFIED DEVELOPMENT LAUNCHER

One command, four services.

A unified Windows developer workflow backed by Docker Compose, without turning the services into a monolith.

REQUIRED PROJECT-ROOT COMMANDS

```
npm run startall
```

START + WAIT

Bring up frontend, API, PostGIS and walking OSRM. Wait for actual readiness, then print working URLs and status.

```
npm run status
```

CHECK

Report each container and endpoint as healthy, unhealthy, unreachable or stopped. Read-only diagnostics.

```
npm run stopall
```

STOP SAFELY

Stop only Near-Circle services. Keep PostgreSQL volumes and prepared pedestrian graphs intact.

STARTUP READINESS CHECKLIST

PostGIS	Database accepts connections
Fastify API	Health endpoint responds
Walking OSRM	Known pedestrian route succeeds
React/Vite web	Frontend is reachable

04 / OPERATIONS + RELIABILITY

Repeatable, safe, and independent.

PowerShell 7 on Windows 11 with Docker Desktop is the target development environment.

01 / Prerequisites

Validate Node/npm, Docker Engine and Compose, configuration, PostGIS availability and a prepared pedestrian graph.

02 / Readiness, not sleep

Respect service dependencies and actual readiness; show actionable errors rather than fixed-delay assumptions.

03 / Project isolation

Configurable, nonconflicting host ports. Bind internal services to the Compose network; never print secrets.

04 / Graph persistence

Bootstrap OSM data once with a documented preparation script. Reuse persisted graphs on ordinary start.

05 / Independent control

Standalone OSRM start, stop, status and direct API tests; checked-in scripts and Compose definitions.

06 / Resilient demo

Unavailable AI key disables only the AI enhancement; core radar, route and steps still run.

07 / External tiles

Do not claim to launch a hosted tile provider. Show configured provider URL and handle tile outages.

08 / Project documentation

Commit .env.example, README instructions, log access, and non-destructive troubleshooting diagnostics.

ACCEPTANCE: STARTUP / SHUTDOWN / REPEATABILITY

A fresh clone + documented bootstrap starts four healthy services. Double-start is safe; status detects stopped/unhealthy services; stop/start preserves data; OSRM works alone; tile outages fail visibly.

Demo-first development phases

Deliver the recognizably useful product first, then add the real geospatial backend and Meta AI enhancement.

1

Mock messenger shell + sample data

React/Vite inbox, 3-5 seeded friends, activation toggle and clearly labeled prototype. Scaffold Compose and launcher.

ACCEPTANCE: fake conversations + initial launcher

2

Radar + consented proximity

PostGIS geography, radius and eligibility queries, radar rings, red dots, friend click and visibility toggle.

ACCEPTANCE: opted-in visible, hidden absent

3

Walking roads, map + steps

Dedicated OSRM foot graph, route API adapter, OSM-compatible tiles, distance, ETA, stride estimator and graph bootstrap.

ACCEPTANCE: walking route + no-route handling

4

Near-Circle API + validation

Fastify schemas, scope enforcement, freshness checks, route authorization, health endpoints and launcher tests.

ACCEPTANCE: API and permission tests

5

Meta AI enhancement

Use the Meta Model API for one permissioned meetup suggestion or concise route-based summary.

ACCEPTANCE: genuine model-backed AI feature

6

Optional Facebook Login + submit

Use an approved auth flow if possible; keep simulation fallback, publish code, provenance and demo evidence.

ACCEPTANCE: judge-ready public submission

Demo priority: one user -> radar -> one friend -> walking route -> step estimate. Production Facebook approval must not block judging.

An honest, scoped demo API.

Sample friends and coordinates are seeded, but the backend must enforce consent, eligibility and route permissions on every request.

METHOD / ENDPOINT	PURPOSE
`GET /demo/friends`	Clearly labeled mock friends; no Meta Graph API claim.
`PUT /near-circle/visibility`	Enable / disable discovery and friend visibility.
`PUT /location`	Update simulated or permissioned user location.
`GET /friends/nearby?radius_m=...`	Eligible opted-in friends; distance and bearing.
`GET /friends/:id/route`	Recheck consent. Walking GeoJSON, distance_m and duration_s.
`GET /friends/:id/steps`	Estimate steps from actual route distance and stride.
`POST /circles/invite`	App-only mock invitations / relationships.
`POST /ai/meetup-suggestion`	Minimal, permissioned context to Meta Model API.
`POST /auth/facebook (optional)`	Approved login if available; never block mock demo.

PROJECT-OWNED DATA MODEL

users	id / display_name / demo_flag
relationships	user_id / friend_id / status
locations	user_id / geography_point / updated_at / expires_at
visibility	user_id / enabled / radius_m / allowed_friends
stride_settings	user_id / stride_m

07 / DEMO + SUBMISSION

Judge-ready presentation

A complete user experience, with a clear line between working back-end logic and simulated social data.

LIVE DEMO SCRIPT / ~2 MINUTES

00:00	**Open prototype**	Explain the unofficial Messenger-style concept.
00:15	**Toggle discovery**	Enable Near-Circle and seeded opted-in friends.
00:35	**Launch radar**	User at center; eligible friend dots appear.
00:55	**Select a friend**	Show identity, consent and approximate proximity.
01:15	**View walking route**	Draw actual walking geometry with ETA and steps.
01:45	**Ask Meta AI**	Show real model-backed suggestion / route summary.
02:10	**Prove new work**	Show Git commits, README and live API / data flow.

HACKATHON PROVENANCE

- Create and version the standalone project in its own public repository.
- Clearly identify all code and features completed during the hackathon.
- Document third-party dependencies, properly licensed OSM data and attribution.
- Disclose any pre-event work; do not present simulated social records as Facebook data.

Demo app is NOT Facebook Messenger; optional login requires approved access.

Definition of done

The end result is independently deployable, demonstrable and explicit about permission and simulation boundaries.

☐ **Consent**	Disabled or unapproved friends never appear in radar or route APIs.
☐ **True proximity**	Radar dots reflect PostGIS distance and bearing, not random layouts.
☐ **Walking route**	Selected friend produces authorized street geometry from a foot-profile graph.
☐ **Visible basemap**	Street tiles load from a configurable OSM-compatible provider with attribution.
☐ **Service isolation**	Frontend, Fastify, PostGIS and OSRM are separately manageable local services.
☐ **External tiles**	No unnecessary fifth local tile container; tile provider is configurable.
☐ **Router autonomy**	OSRM has persisted foot graph, API, health checks and solo lifecycle.
☐ **Unified launcher**	startall, status and stopall manage Near-Circle safely with real readiness checks.
☐ **Persistence**	Ordinary restart does not delete database data or rebuild the pedestrian graph.
☐ **Fresh environment**	Project starts independently from its own source, config and documented bootstrap.
☐ **Estimates**	Steps = round(route_m / stride_m); default stride is 0.75 m.
☐ **Genuine Meta AI**	One real model-backed feature; unavailable API degrades safely.
☐ **Compliance**	Public submission distinguishes hackathon-built work, simulation and prior tooling.

TECHNICAL REFERENCES

OSRM profiles:	https://github.com/Project-OSRM/osrm-backend/blob/master/docs/profiles.md
OSRM foot profile:	https://github.com/Project-OSRM/osrm-backend/blob/master/profiles/foot.lua
OSM tile usage:	https://operations.osmfoundation.org/policies/tiles/